Most Asked JavaScript Interview Questions With Solutions & Explanations

1. What is Hoisting?

```
Answer:
Hoisting moves declarations to the top during execution.

Example:

console.log(a);

var a = 10;

Explanation:
var is hoisted with undefined.
let and const stay in Temporal Dead Zone.
```

2. Difference Between null and undefined

```
undefined:
Variable declared but no value assigned.

null:
Intentional empty value.

Example:

let a;
console.log(a);

let b = null;
```

3. Difference Between == and ===

```
==
Checks value only.

===
Checks value and datatype.

Example:

5 == '5'   // true
5 === '5'  // false
```

4. What is Event Delegation?

```
Answer:
Handling child events using parent listener.

Example:

parent.addEventListener("click", (e) => {
  if(e.target.tagName === "BUTTON") {
    console.log("clicked");
  }
});

Explanation:
Improves performance.
```

5. Explain Event Bubbling and Capturing

```
Bubbling:
Child to parent.

Capturing:
Parent to child.

Example:

div.addEventListener("click", fn, true);

true enables capturing.
```

6. Difference Between map, filter, and reduce

```
map:
Transforms array.

filter:
Returns matching elements.

reduce:
Returns single value.

Example:

arr.map()
arr.filter()
arr.reduce()
```

7. Difference Between Synchronous and Asynchronous Code

```
Synchronous:
Executes line by line.

Asynchronous:
Does not block execution.

Example:

setTimeout(() => {
  console.log("Hello");
}, 1000);
```

8. Explain Promises and async/await

```
Promise:
Handles async operations.

Example:

fetch(url)
  .then(res => res.json())

async/await:

async function getData() {
  const data = await fetch(url);
}
```

9. What is Callback Hell?

```
Nested callbacks causing unreadable code.

Example:
login(() => {
```

```
  getData(() => {
    updateUI();
  });
});
```

```
Solution:
Use promises or async/await.
```

10. What is Debouncing and Throttling?

```
Debouncing:
Runs after delay.

Throttling:
Runs at intervals.

Used in:
- Search bars
- Scroll events
```

11. Difference Between localStorage and sessionStorage

```
localStorage:
Permanent storage.

sessionStorage:
Clears after tab close.
```

12. Explain this Keyword

```
this refers to current execution context.

Example:

const user = {
  name: "Arun",
  greet() {
    console.log(this.name);
  }
};
```

13. Difference Between call, apply, and bind

```
call:
Arguments separately.

apply:
Arguments in array.

bind:
Returns new function.

Example:

fn.call(obj, 1, 2);
fn.apply(obj, [1,2]);
```

14. What are Pure Functions?

```
Pure functions:
- Same input gives same output
- No side effects

Example:
function add(a,b) {
```

```
  return a+b;
}
```

15. Explain Prototype Inheritance

```
JavaScript uses prototype-based inheritance.

Objects inherit properties from prototypes.

Example:

Array.prototype
```

16. What is Memoization?

```
Stores previous calculations.

Improves performance.

Used in:
- Expensive calculations
- React optimization
```

17. How JavaScript Execution Context Works?

```
Two phases:
1. Memory creation
2. Code execution

Functions and variables stored in memory first.
```

18. Difference Between Shallow Copy and Deep Copy

```
Shallow copy:
Copies reference.

Deep copy:
Copies nested data completely.

Example:

const copy = {...obj};
```

19. Explain Spread and Rest Operators

```
Spread:
Expands array/object.

Rest:
Collects values.

Example:

const arr2 = [...arr1];

function sum(...nums) {}
```

20. What are Higher Order Functions?

```
Functions that:
- Take function as argument
OR
```

```
- Return function

Examples:
map
filter
reduce
```

21. Difference Between forEach and map

```
forEach:
No return.

map:
Returns new array.

Example:

arr.map(item => item * 2);
```

22. What is Currying?

```
Converting function into multiple smaller functions.

Example:

function add(a) {
  return function(b) {
    return a+b;
  }
}
```

23. How Event Loop Works?

```
Event loop handles async tasks.

Flow:
Call Stack -> Web APIs -> Callback Queue -> Event Loop
```

24. What is Temporal Dead Zone?

```
TDZ is the time before let/const initialization.

Example:

console.log(a);

let a = 10;
```

25. What Causes Memory Leaks?

```
Common reasons:
- Unremoved event listeners
- Global variables
- Timers not cleared
```

26. Difference Between Function Declaration and Expression

```
Declaration:
Hoisted fully.

Expression:
Not fully hoisted.
```

```
Example:

function test(){}

const test = function(){}
```

27. What are Generators?

```
Generators pause execution using yield.

Example:

function* gen() {
  yield 1;
}
```

28. Explain Microtasks and Macrotasks

```
Microtasks:
Promises.

Macrotasks:
setTimeout.

Microtasks execute first.
```